

- a) Diagrama de los conceptos: El PDF se encuentra en el repositorio bajo el nombre Entregable1_a_Diagrama_Conceptos.pdf.
- b) Estructuras de datos propuesta:

Funcionalidad 1: Reportar si hay más de dos sistemas autónomos reportando un prefijo como propio.

Para implementar esta funcionalidad es importante saber que al tener un prefijo dado, solo debería haber un AS de origen quien anuncie dicho prefijo, en caso contrario, si hubiese dos o más AS anunciándolo sería una fuerte sospecha de prefix hijacking y por ende es una anomalía que se debe detectar y reportar.

Teniendo esto en cuenta junto a estructura del archivo MRT, basta con que para cada registro en el que aparece un determinado prefijo, se identifique el último AS del AS path y si un prefijo posee dos o más AS de origen sería sospechoso y corresponde notificarlo.

La estructura de datos adecuada para esto es un diccionario que use como clave el prefijo y guarde los AS de origen únicos, esto es importante para no repetir cientos de veces el mismo AS de origen dentro del diccionario porque sería un desperdicio de espacio y no es de interés tener AS de origen repetidos ni el resto del AS path para la solución de este problema.

Con esta estructura es muy fácil y eficiente verificar el objetivo de la funcionalidad: detectar prefijos que son reclamados por diferentes AS de origen, debido a que es suficiente con conocer la cantidad de elementos asociados a cada clave.

Ejemplo de la estructura:

```
as_origen_prefijo = {  
    "83.68.192.0/19": {44885},  
    "83.69.0.0/19": {21191},  
    "163.178.0.0/16": {11816},  
    "8.8.8.0/24": {15169, 64512}  
}
```

En este ejemplo el caso a reportar sería el del prefijo "8.8.8.0/24": porque cuenta con dos AS de origen.

Funcionalidad 2: Consulta de un Sistema Autónomo (AS)

Para implementar la segunda funcionalidad se propone utilizar un diccionario anidado, cuya clave principal corresponde al número del Sistema Autónomo (AS). Esta estructura permite organizar la información de acuerdo con el AS consultado y acceder directamente a sus datos, sin necesidad de recorrer todos los registros almacenados. Para cada AS se

almacenará otro diccionario que tendrá como clave la dirección IP del peer, la cual representa una de las aristas asociadas a ese sistema autónomo.

La estructura propuesta se puede representar de la siguiente manera:

```
datos_por_as = {  
  53046: {  
    "177.101.16.80": [  
      {  
        "prefijo": "7.0.0.0/8",  
        "as_path": [53046, 61626, 14840, 3356, 749]  
      }  
    ]  
  }  
}
```

En este ejemplo, 53046 corresponde al número del AS que funciona como llave principal del diccionario. Dentro de este se encuentra la dirección IP 177.101.16.80, correspondiente al peer IP o arista asociada con ese AS. Para cada arista se utiliza una lista, debido a que un mismo AS puede conocer varios prefijos a través de un mismo peer.

Cada elemento de la lista almacena el prefijo de destino y su respectivo AS Path. Por ejemplo, el prefijo 7.0.0.0/8 tiene asociado el camino [53046, 61626, 14840, 3356, 749], el cual representa la secuencia de sistemas autónomos que conforman esa ruta. De esta manera, si existen varios prefijos conocidos mediante una misma arista, estos pueden agregarse como nuevos elementos dentro de la lista.

La principal ventaja de esta estructura es que se adapta directamente a la consulta solicitada por la funcionalidad 2: AS → aristas → prefijos → AS Path. Al utilizar diccionarios, la aplicación puede localizar directamente un AS mediante su número y, a partir de este, obtener las aristas asociadas y la información de los prefijos conocidos por cada una. Esto evita tener que recorrer toda la estructura de datos cada vez que se realiza una consulta y facilita la presentación tabulada de los resultados solicitados.

Funcionalidad 3: Todas las rutas disponibles entre A y B

Para esta funcionalidad, donde se debe mostrar todas las rutas disponibles que existen entre un prefijo llamado A y otro prefijo llamado B, primero se debe encontrar qué sistemas autónomos originan el prefijo A y cuales originan el prefijo B. Luego de esto ya se encuentran todas las secuencias de sistemas autónomos que conectan cada uno, en este caso concreto no se debe filtrar la ruta más corta, sino mostrar todas las rutas.

Para definir la estructura de los datos se puede utilizar una lógica similar a los grafos, ya que representa los caminos o rutas que existen entre un lugar y otro. Y la forma de poder representar un grafo es nuevamente utilizando un diccionario anidado.

Por ejemplo, si tenemos este prefijo 151.159.192.0/20, que lo origina el sistema autónomo 6057, y 151.183.32.0/20 lo origina el sistema autónomo 24158. Supongamos que en el dump aparecen estos AS Paths entre ambos:

6057 3356 9924 24158

6057 1299 24158

6057 3356 1299 24158

Por lo cual el diccionario anidado queda de esta forma:

rutas = {

 "151.159.192.0/16-151.183.32.0/20": {6057 3356 992 24158, 6057 129 24158, 6057 3356 1299 24158}

}

De este modo mediante una llave de los dos sistemas autónomos se visualizan todas las rutas posibles.